



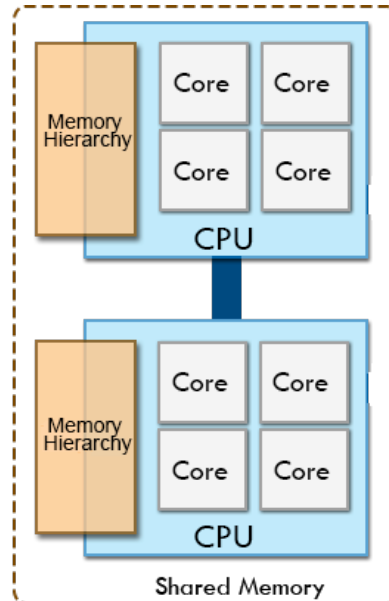
Development on Heterogeneous Systems

2025

André Pereira

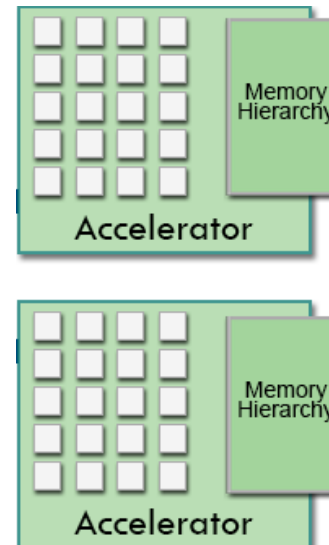


- CPUs have multiple complex computing cores
 - Use processes/threads to parallelise the code



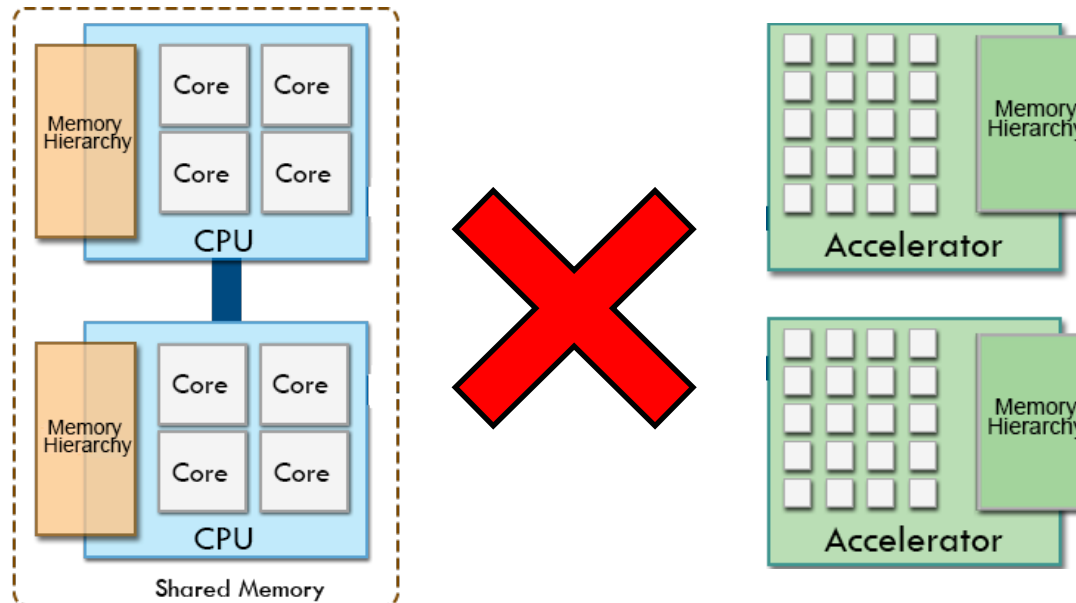


- CPUs have multiple complex computing cores
 - Use processes/threads to parallelise the code
- GPUs support a very high number of simultaneous threads
 - Offload data intensive computations to the device



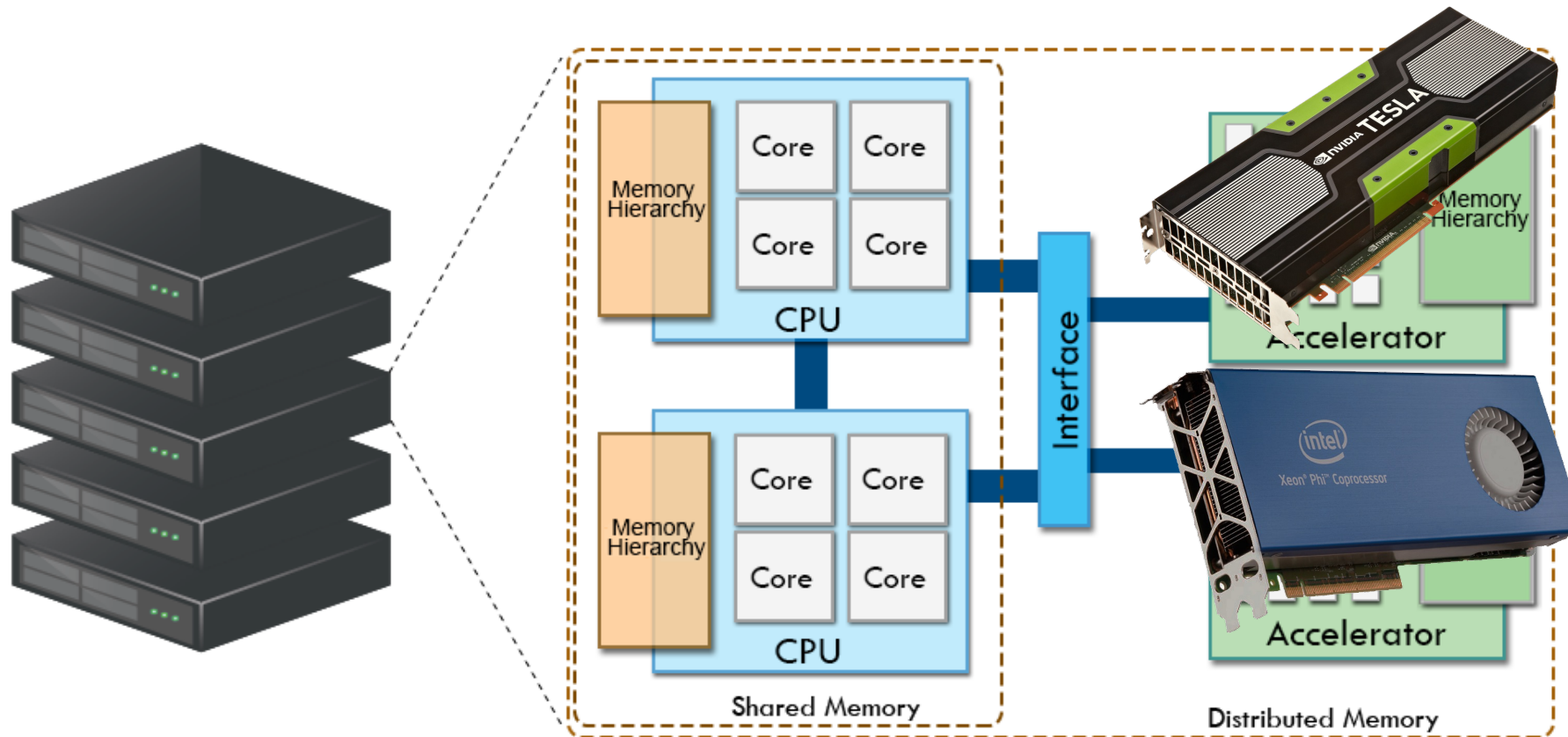


- Both devices coexist on the same system, but...
 - The processing power of the CPU is wasted when offloading code to the GPU, and vice-versa
- Why not simultaneously use both devices?



Motivation *HetPlat Architecture* Heterogeneous Platforms (HetPlats)

Levels of Parallelism
Performance Scalability
Performance Portability



Motivation

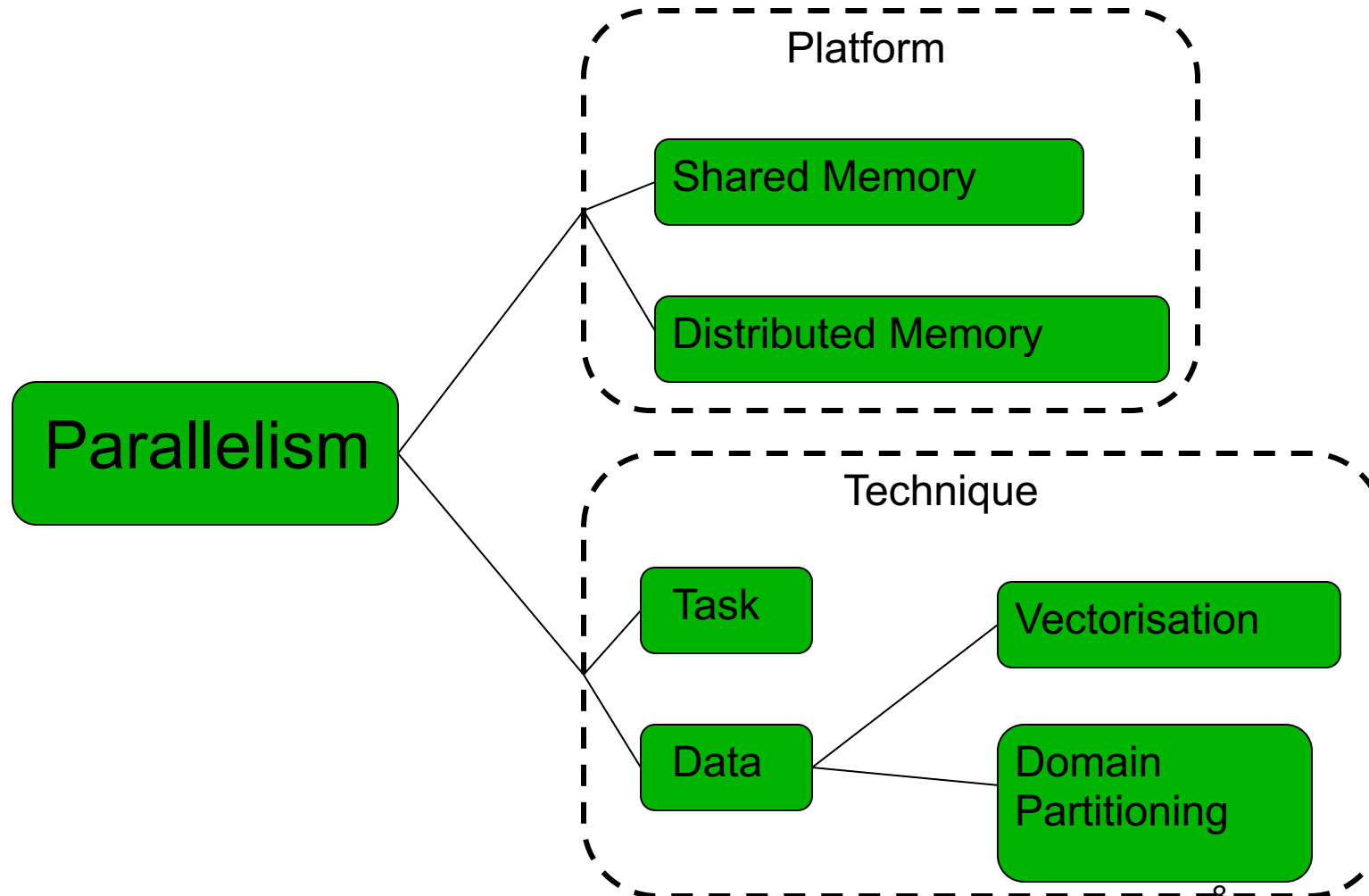
Heterogeneous Platforms (HetPlats)

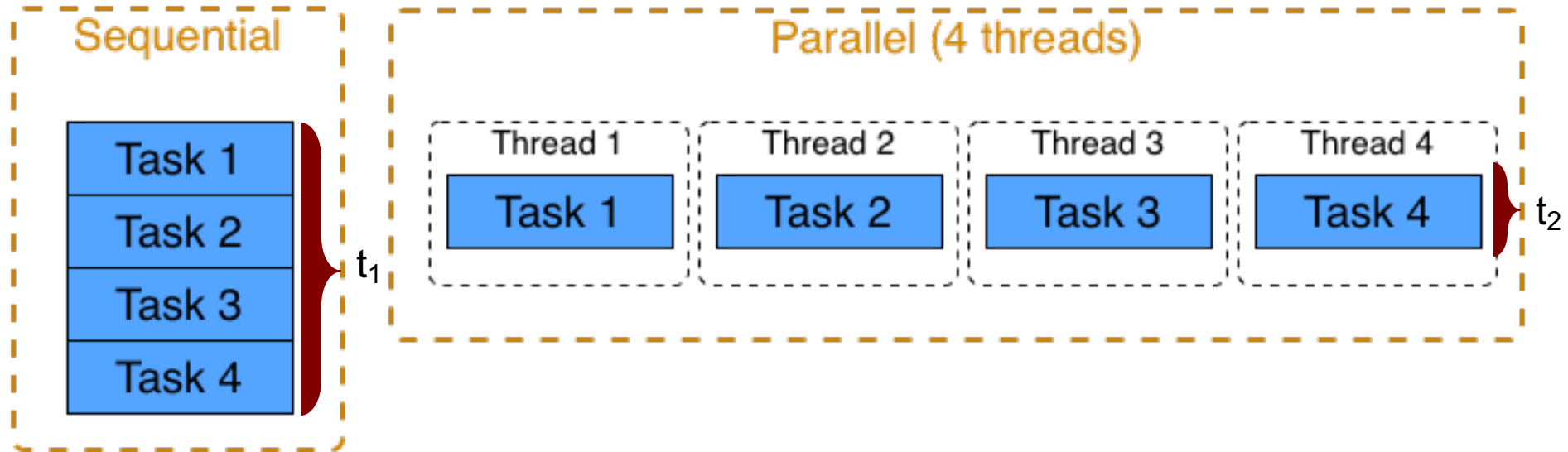
Levels of Parallelism

Levels of Parallelism

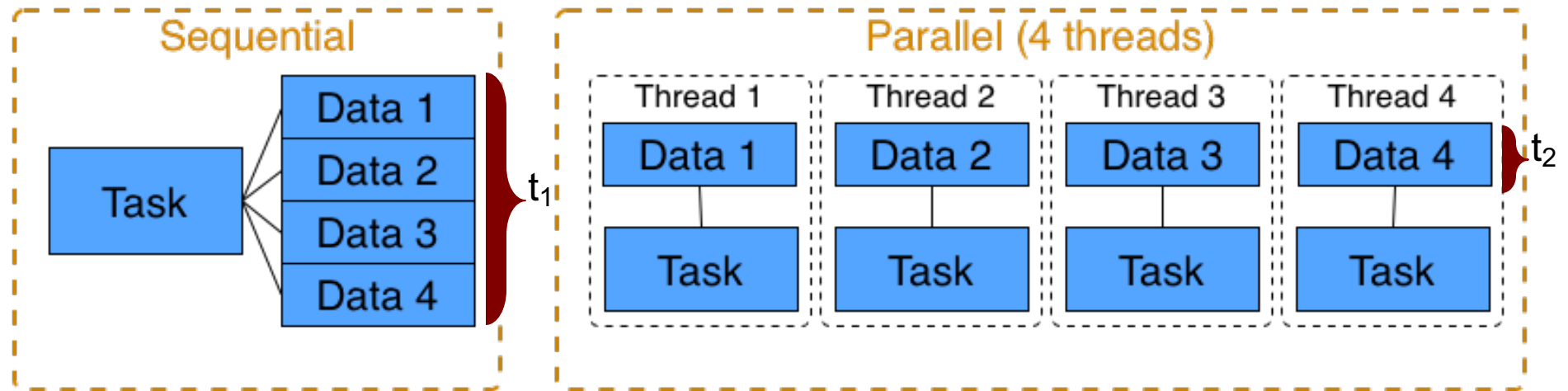
Performance Scalability

Performance Portability





$$t_2 \approx t_1 / 4$$



$$t_2 \approx t_1 / 4$$



Sequential

```
void stencil_1d(int *in, int *out) {  
    int result = 0;  
  
    for (int i = 0; i < SIZE; i++) {  
        for (int offset = -RADIUS; offset <= RADIUS; offset++)  
            result += in[i + offset];  
        // Store the result  
        out[i] = result;  
    }  
}
```

Parallel - CPU

```
void stencil_1d(int *in, int *out) {  
    int result = 0;  
    #pragma omp parallel for  
    for (int i = 0; i < SIZE; i++) {  
        for (int offset = -RADIUS; offset <= RADIUS; offset++)  
            result += in[i + offset];  
        // Store the result  
        out[i] = result;  
    }  
}
```



Parallel - CPU

```
void stencil_1d(int *in, int *out) {  
    int result = 0;  
    #pragma omp parallel for  
    for (int i = 0; i < SIZE; i++) {  
        for (int offset = -RADIUS; offset <= RADIUS; offset++)  
            result += in[i + offset];  
        // Store the result  
        out[i] = result;  
    }  
}
```

Parallel - GPU

```
void stencil_1d(int *in, int *out) {  
    int result = 0;  
    int i = threadIdx.x + blockIdx.x * blockDim.x;  
  
    for (int offset = -RADIUS; offset <= RADIUS; offset++)  
        result += in[i + offset];  
    // Store the result  
    out[i] = result;  
}
```

However, it is highly inefficient...



GPU – Optimised

```
void stencil_1d(int *in, int *out) {
    int result = 0;
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + RADIUS;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }
    // Synchronize (ensure all data is available)
    __syncthreads();

    for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
        result += temp[i + offset];
    // Store the result
    out[i] = result;
}
```

With simple optimisations the code complexity starts to increase considerably....

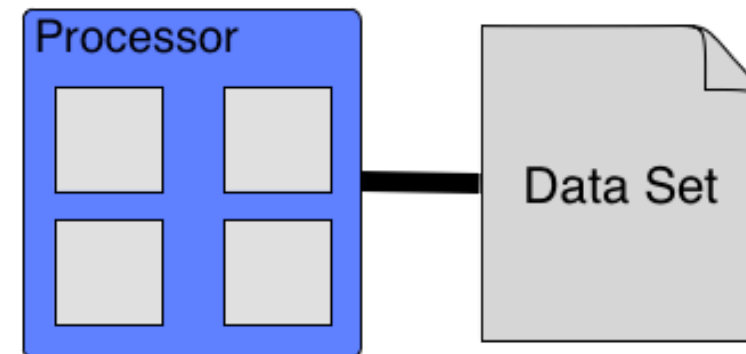
Challenges in Heterogeneous Computing

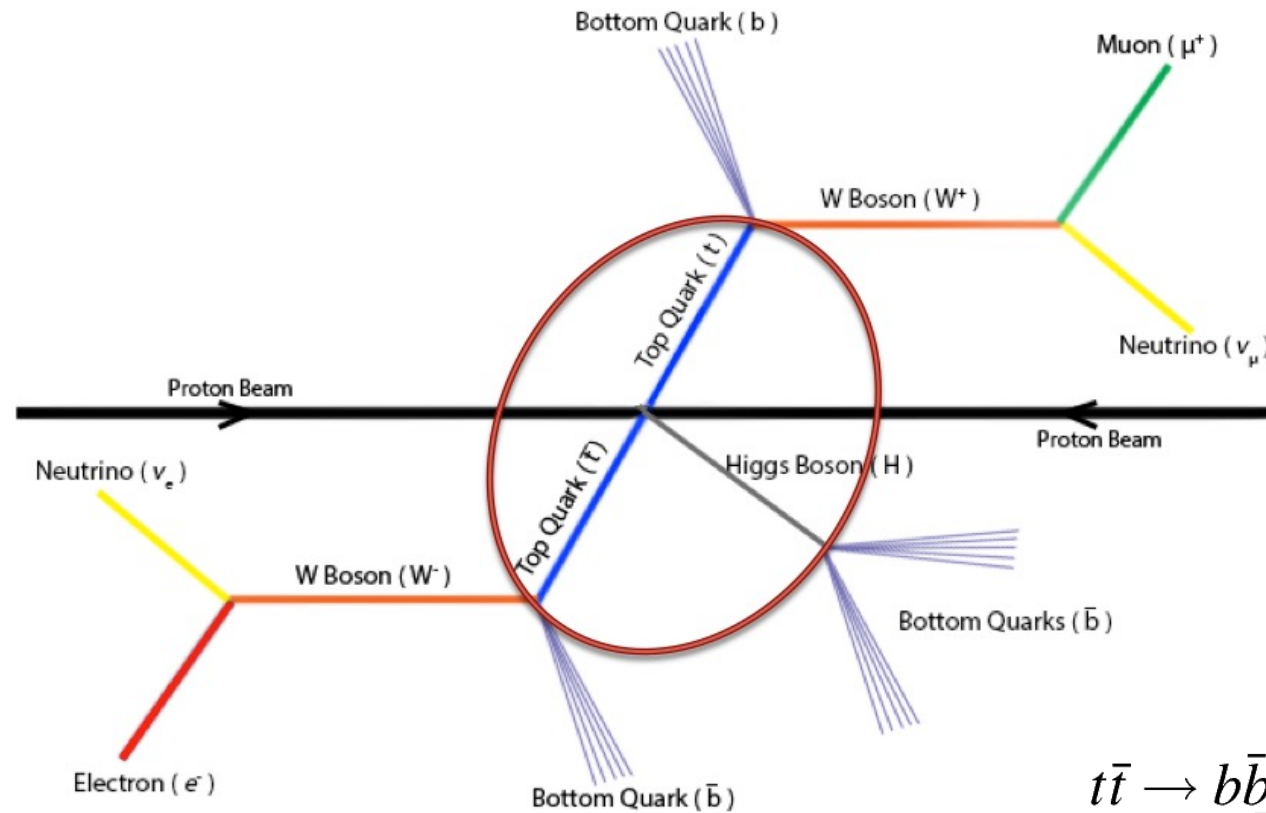


- Different architectures
 - Distinct designs of parallelism
 - Distinct memory hierarchies
- Different programming paradigms
 - Distinct code for efficient algorithms among devices
- Workload management
 - High latency communication between CPU and device
 - Different throughputs among devices



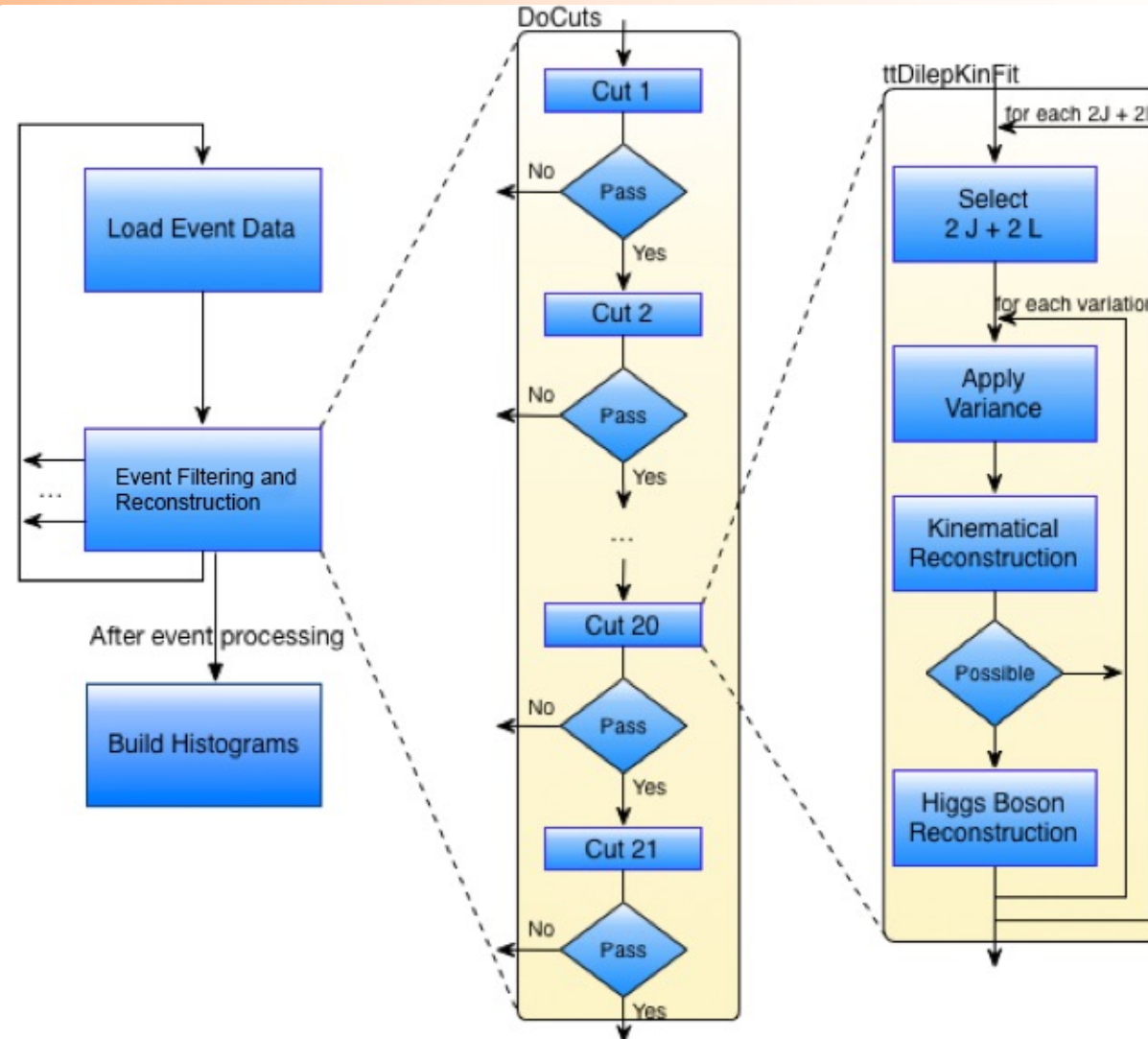
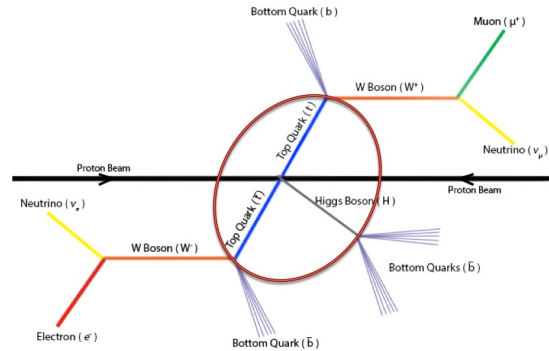
- Consider an efficient algorithm, with optimised parallel code
- Does the performance scale?
 - With the data set size
 - **With the increased number of cores**





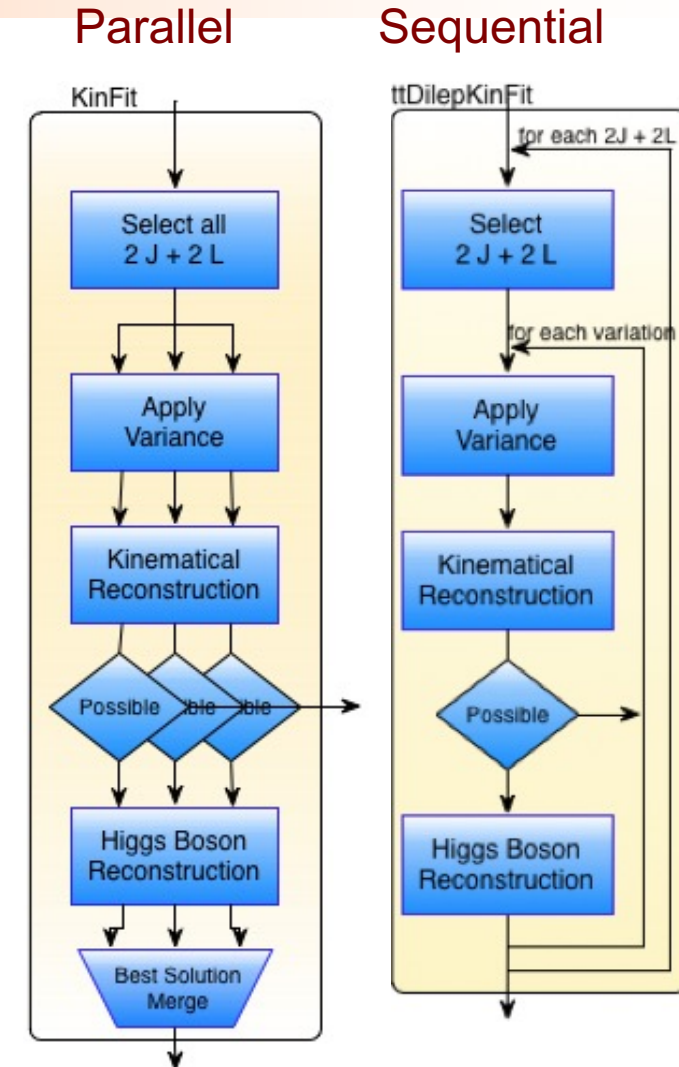
$$t\bar{t} \rightarrow b\bar{b}l^+l^- \nu\bar{\nu}$$

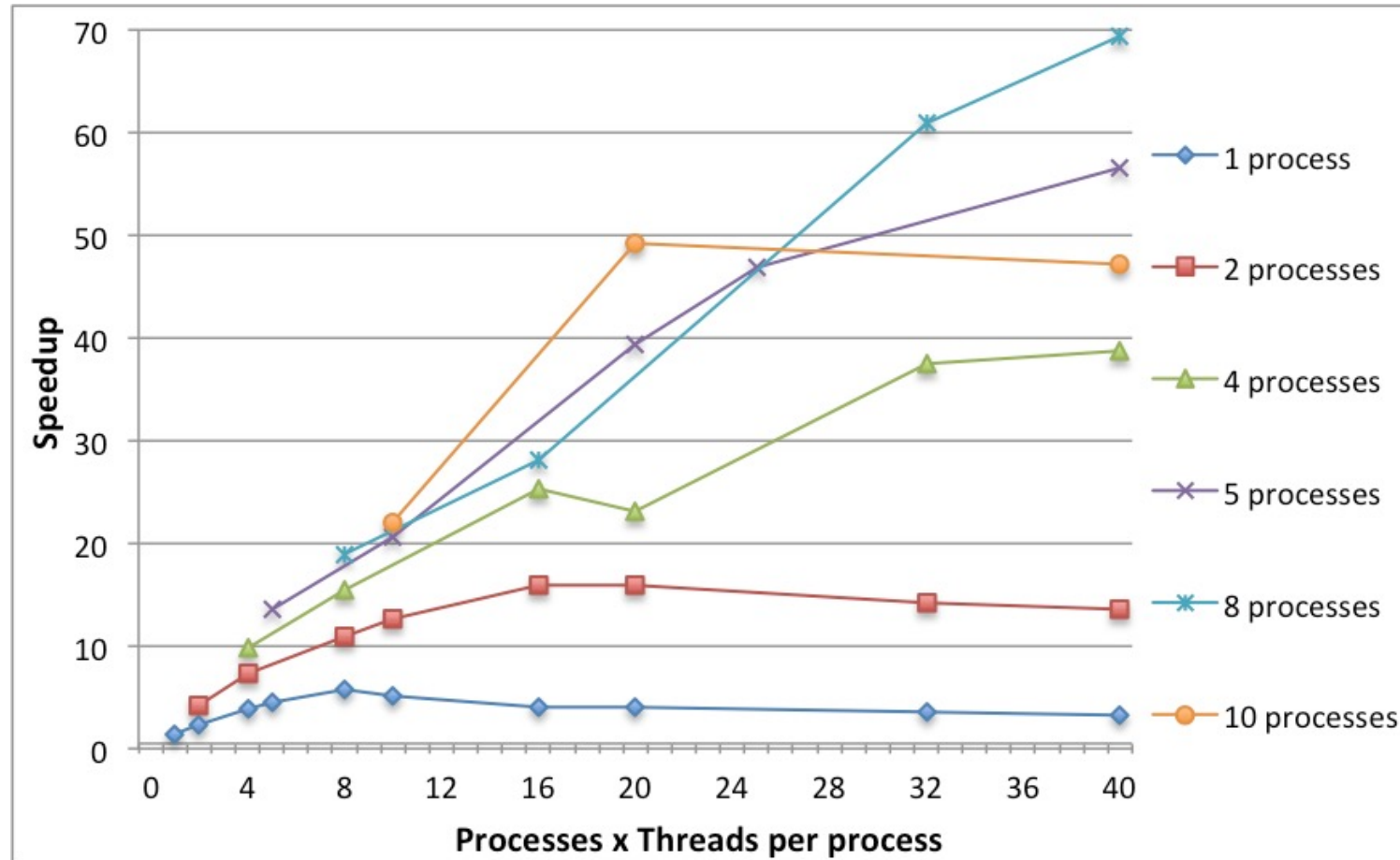
$$H \rightarrow b\bar{b}$$





- Optimisation on a dual socket homogeneous system
 - KinFit takes 99.8% overall execution time
- Execution Parallelism
 - Exploring multi-core devices
- Runtime inefficiencies
 - Multithreading inefficiencies





2x 10-core Intel Xeon E5-2670v2, with Hyperthreading, 64GB RAM



- The code either uses the CPU or the GPU
 - Each requires different code
- Either CPU or GPU processing... Only a factor of the system processing potential is being used
 - It would be great to get both codes to simultaneously work sharing the same data set



Motivation

Heterogeneous Platforms (HetPlats)

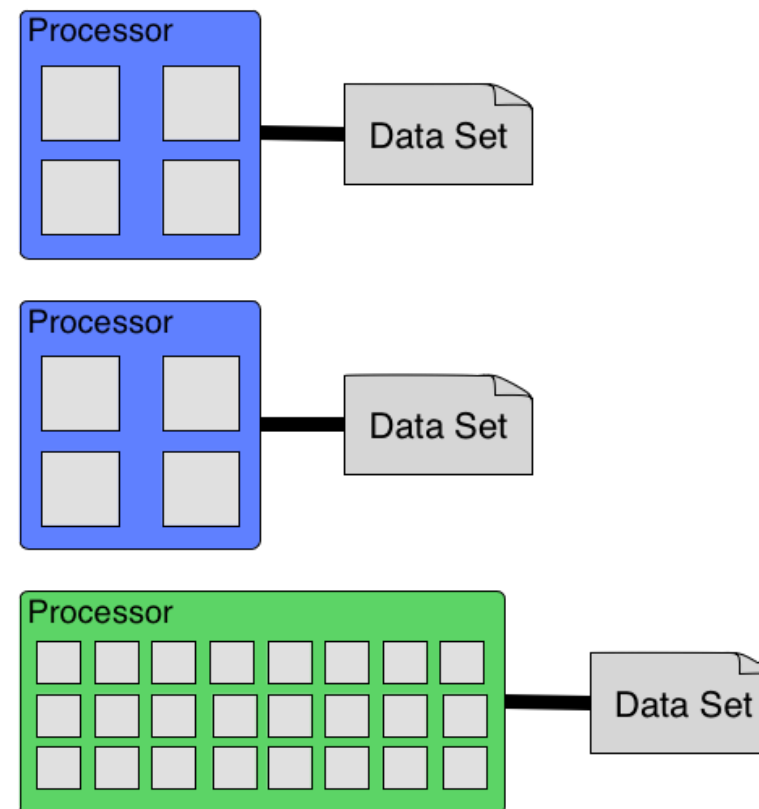
Levels of Parallelism

Performance Scalability

Performance Portability

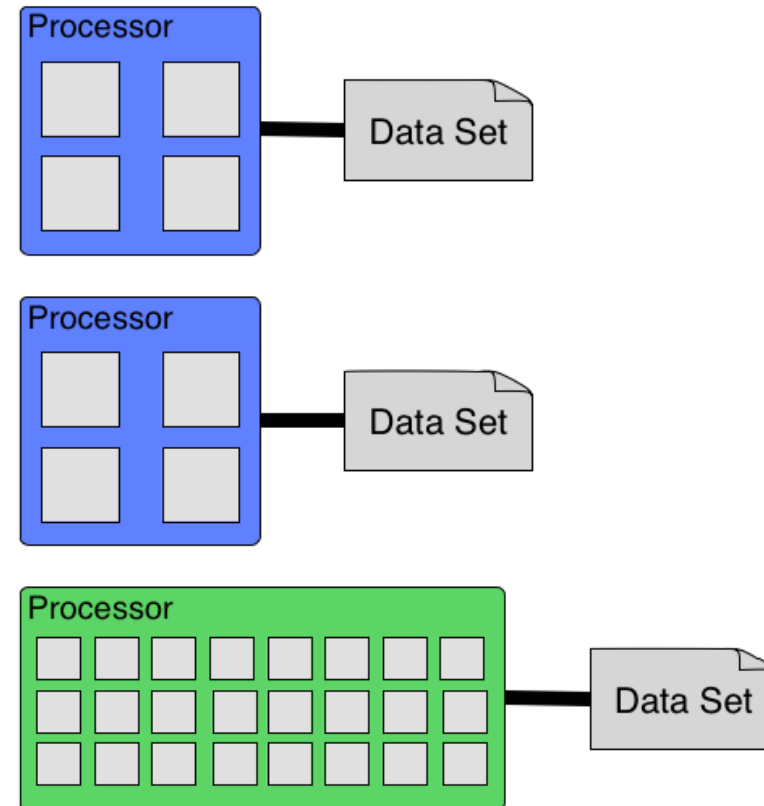
Performance Portability

- Several challenges arise
 - How is data partitioned?
 - How is data balanced among devices?
 - The code needs synchronisation?
 - Will the code scale?



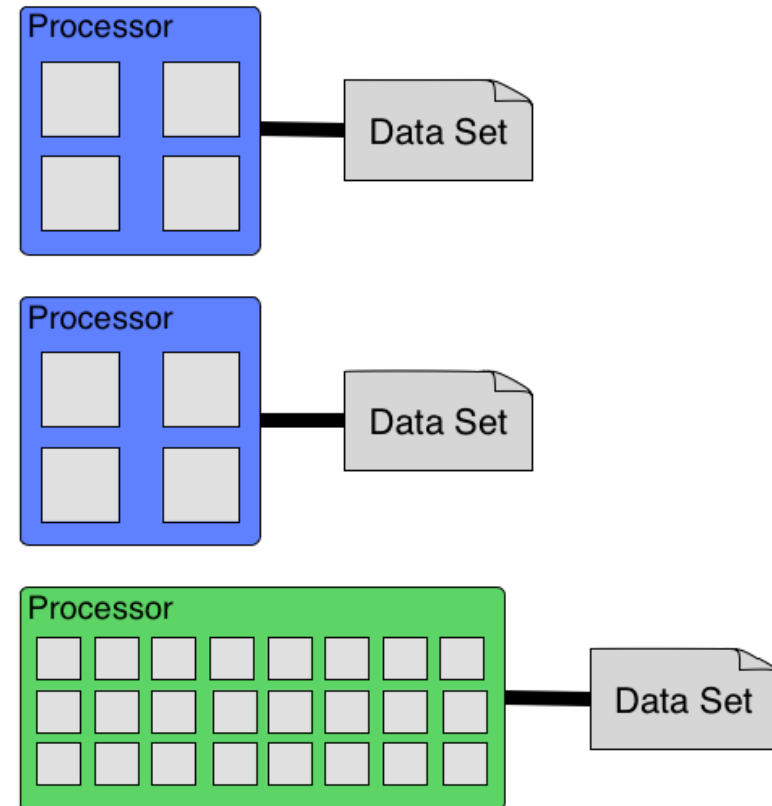


- Requires very complex coding
 - Code the algorithm for the CPU
 - Efficiently manage the parallel workload
 - Ensure its performance scalability
 - Code the algorithm for the accelerator
 - Efficiently manage the parallel workload
 - Ensure its performance scalability



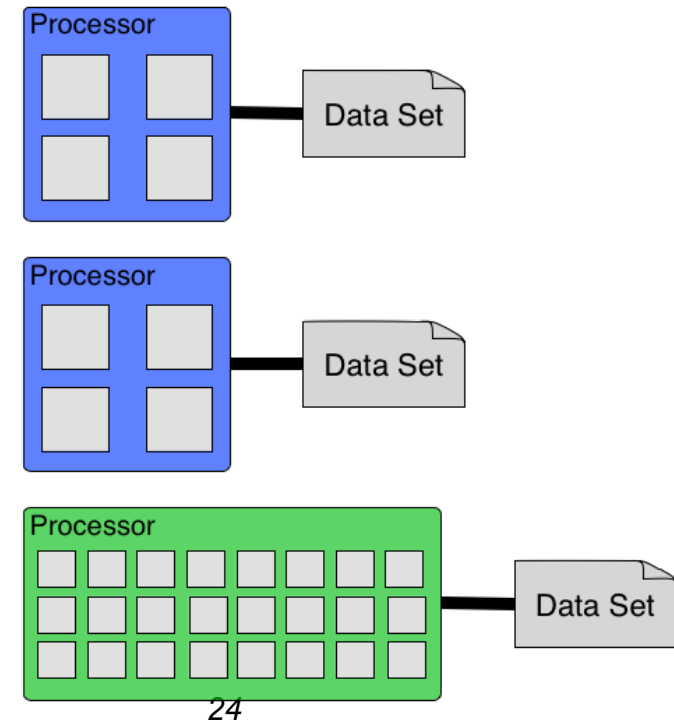


- Requires very complex coding
 - Manage the workload among different devices
 - Not trivial to code
 - Data transfers use a low bandwidth connection
 - Computing devices have different processing throughputs
 - What is the best data chunk size for each?
 - What is the best scheduling technique?





- “I spent months optimising my code for HetPlats, I bet it will be super fast on this new system I just bought”
 - No! You need to re-tune the code for each system...
- How is it possible to
 - achieve code scalability in each device?
 - simultaneously use both computing devices?
 - write the code once and guarantee its performance across different HetPlats?

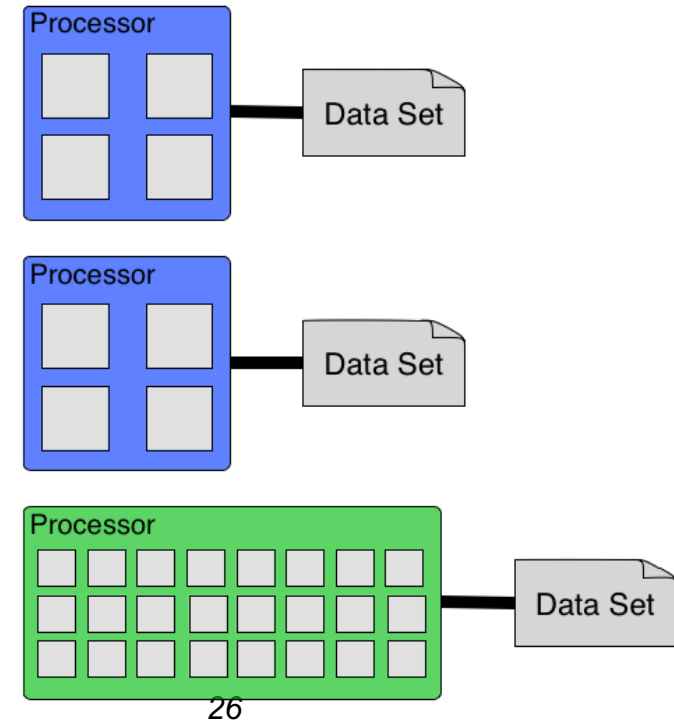


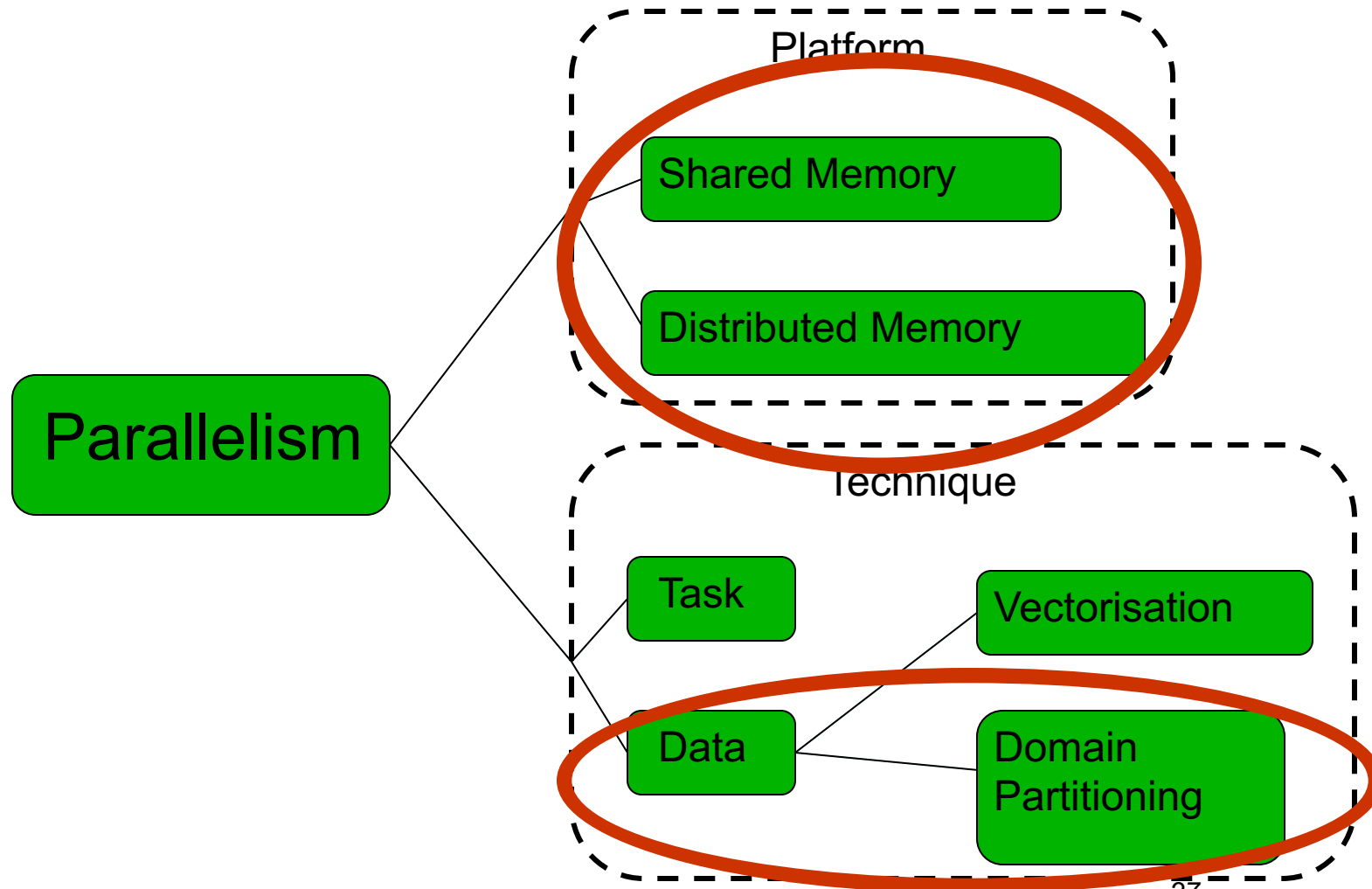


- Current computing platforms are heterogeneous
 - Multicore CPUs coupled with GPUs
 - Current parallel code uses either the CPU or the GPU
- Performance scalability is not linear
 - Having more cores does not always mean faster code
- Performance portability is very complex to achieve
 - Each different computing platform requires specific hand tuning
 - Workload balancing is very complex when using CPUs and GPUs simultaneously



- “I spent months optimising my code for HetPlats, I bet it will be super fast on this new system I just bought”
 - No! You need to re-tune the code for each system...
- How is it possible to
 - achieve code scalability in each device?
 - simultaneously use both computing devices?
 - write the code once and guarantee its performance across different HetPlats?







- There are frameworks to help the development of code for heterogeneous platforms
- They provide several key features to the programmer
 - Abstraction of the distributed memory environment
 - Automatic workload balance among processing units
 - Coding the algorithm once to run on different processing units
 - Management of different task dependencies
 - Adaptation to the computing platform
- They are open source!
 - And provide several tutorials



- The downside is...
 - Steep learning curve for non-computer scientists
 - Production code has to be re-written to fit their programming model
 - Some frameworks require user configuration for each task/algorithm, which may have a huge impact on performance
- Different frameworks use different strategies to
 - Implement the algorithms
 - Minimise the costs of transferring the data among processing units
 - Handle **RAW**, **WAW**, and **WAR** task dependencies
 - Schedule the workload among processing units

Revisiting the Challenges



- Different architectures ✓
 - Distinct designs of parallelism
 - Distinct memory hierarchies
- Different programming paradigms ✓
 - Distinct code for efficient algorithms among devices
- Workload management ✓
 - High latency communication between CPU and device
 - Different throughputs among devices



- “*Task programming library for hybrid architectures*”
- Implementation through the library API or compiler pragmas
- Uses a task-based parallelisation approach
 - Programmer codes *codelets* to run on the processing units
 - StarPU hides memory transfer costs by interleaving different tasks
 - Fixed workload grain size (defined by the user)
 - Also works on cluster environments with MPI



- *“Data-centric programming model for writing high performance applications”*
- A parallelisation approach focused on the data set
 - User specifies properties to the data structures, such as organisation, partitioning, and coherence
 - Legion handles the parallelism and data transfer, according to the specified properties
 - User maps the tasks to the processing units
 - Legion schedules the workload at runtime to handle irregular tasks



- Programming model and runtime system for irregular applications
 - **D**ynamic **I**rregular **C**omputing **E**nvironment
- Data parallelism approach with an unified memory space
 - Provides various data containers, with different properties
 - Allows to provide optimised code for each processing unit
 - The user has to code a dicing function – used to minimise the data transfers
 - Workload grain size adapts dynamically at runtime
- Requires some expertise to produce high performing code



- Target C++ programmers (template, lambda)
 - No language extension
 - No pragmas
 - No attribute
- Borrows lot of concept from battle tested OpenCL (platform, device, work-group, range)
- Single Source (two compilation pass)
- Implicit or Explicit data-transfer
- SYCL is a Specification developed by the Khronos Group (OpenCL, SPIR, Vulkan, OpenGL)
- Nice interoperability with other programming model (OpenMP, CUDA, Hip, OpenCL)

